



How does an AI answer know **where on the page** it came from?

Follow one question through PaperLab, from the uploaded PDF to the citation you click.

How is BERT pre-trained?

It masks 15% of the input tokens and learns to predict them **[C1]**.



C1

C2



Swipe →

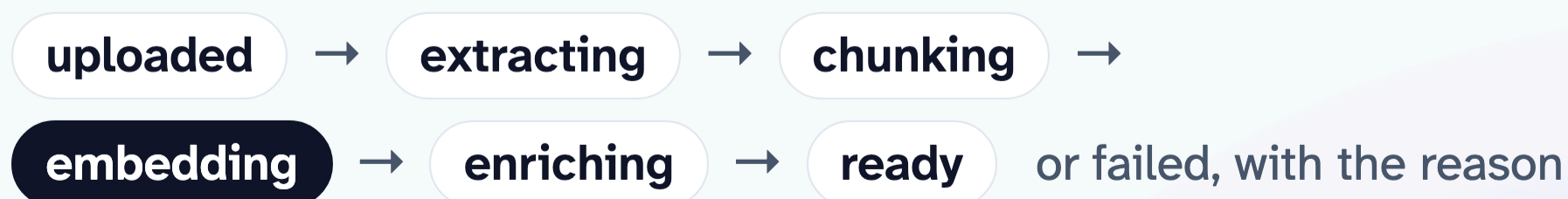
1 • UPLOAD

One background job, safe to run again.

The API stores the PDF, queues a job in Redis and returns right away. An ARQ worker runs it. A re-run replaces what the last one made.



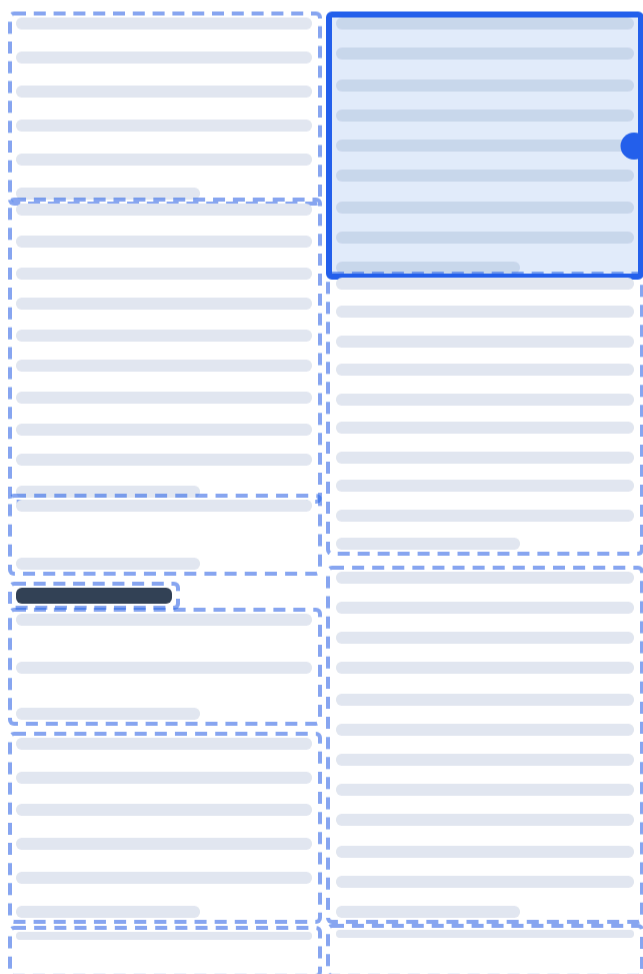
The paper's status



2 • EXTRACT

Text keeps its place on the page.

PyMuPDF reads each page as blocks of text. Every block keeps its page number and its rectangle, in PDF points.

**text**

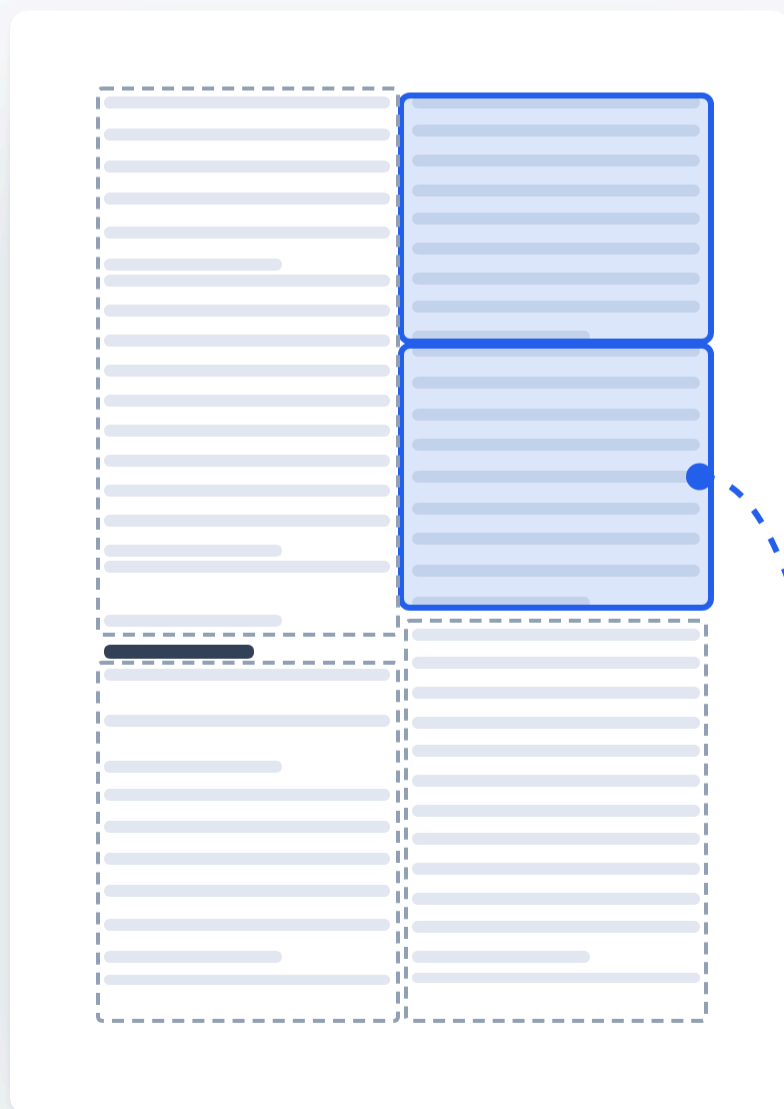
In order to train a deep bidirectional representation, we simply mask some percentage of the input tokens...

page**4****bbox****[307, 65, 526, 253]**

3 · CHUNK

Blocks become chunks, split at headings.

A chunk ends at a heading, a page break or about 1,500 characters. It keeps its page, its section and one rectangle per block.



p. 4 · 3 BERT

p. 4 · 3.1 Pre-training BERT

p. 4 · 3.1 Pre-training BERT

In order to train a deep
bidirectional representation, we
simply mask...

1,367 characters · 2 rectangles

4 • EMBED

768 numbers per chunk, in the same Postgres.

The nomic-embed-text-v1.5 model turns each chunk into a vector. pgvector stores it next to the papers and notes, with an HNSW index.

search_document: In order to train a deep bidirectional representation, we simply mask some...

p. 4 • 3.1 Pre-training BERT

nomic-embed-text-v1.5

[0.018, -0.041, 0.066, -0.009, ...]

768 numbers

Postgres + pgvector

papers

chunks

notes

embedding vector(768) • HNSW index, cosine distance

5 • RETRIEVE

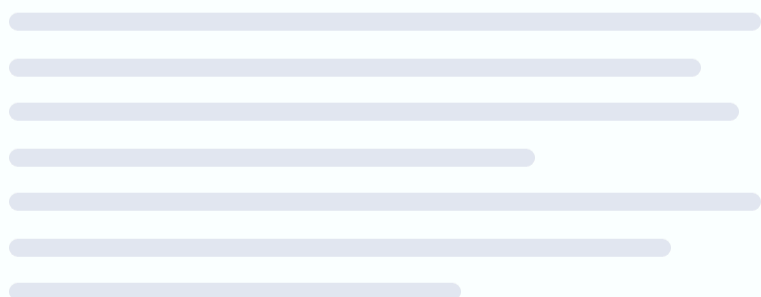
Short paper: all of it. Long paper: 8 chunks.

A paper of up to 24,000 characters is sent whole. A longer one, like BERT, sends the 8 chunks closest to the question.

How is BERT pre-trained?

Up to 24,000 characters

Whole paper • 18 chunks



in reading order

BERT: 60,821 characters

C1 p. 4 • 3.1 Pre-training BERT

C2 

C3 

C4 

C5 

C6 

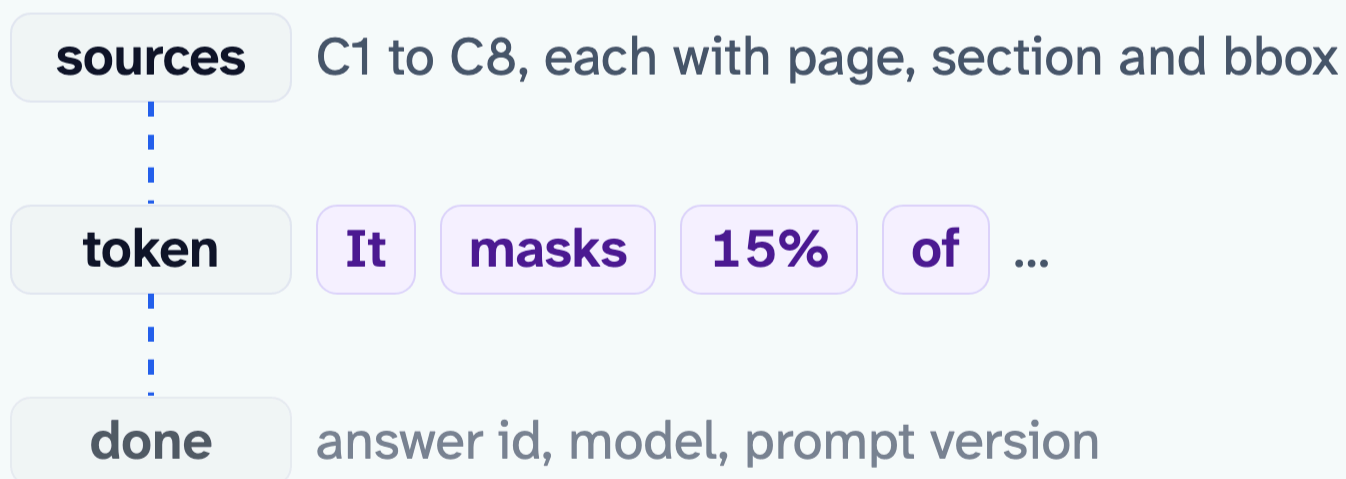
C7 

C8 

6 • STREAM

Sources arrive before the first word.

The answer streams over Server-Sent Events: **sources**, then **token** after **token**, then **done**. So [C1] can be clicked while the answer is still being written.



It masks 15% of the input tokens and learns to predict them [\[C1\]](#)



C1

C2

C3

C4

C5

C6

C7

C8

7 • CHECK

Click [C1]. See where on the page it came from.

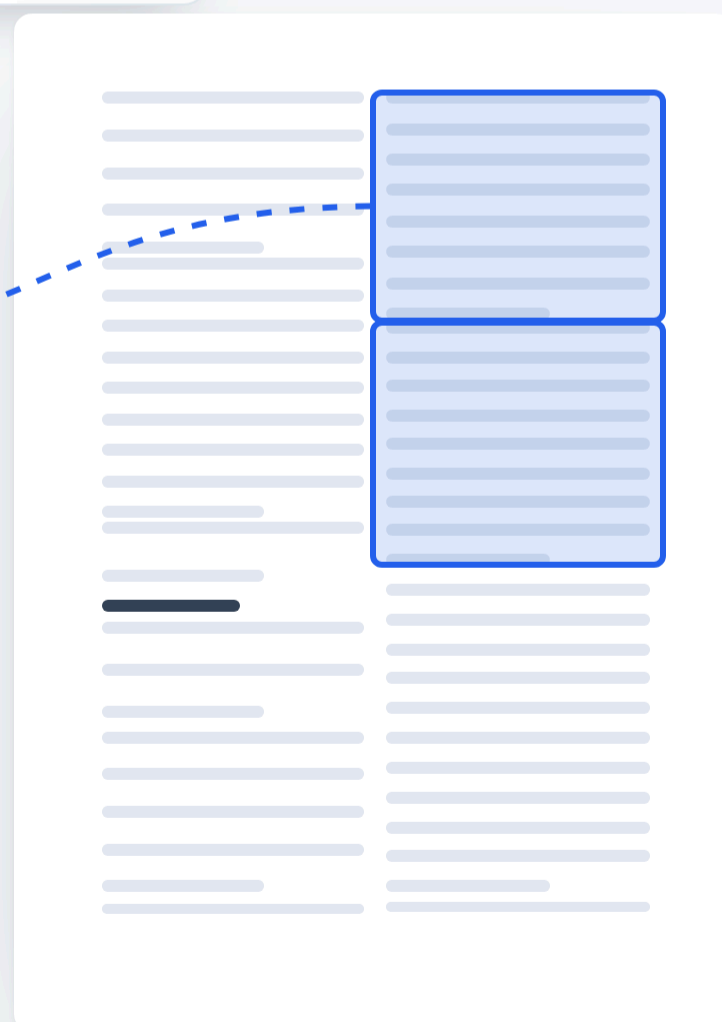
The reader scrolls to page 4 and flashes the chunk's two rectangles. The answer is stored with the model that wrote it and its prompt version.

Source 1: page 4, section “3.1 Pre-training BERT”

It masks 15% of the input tokens
and learns to predict them [C1].

Saved with the answer

☆ AI · qwen3:8b · prompt v2





Read deeper. Keep every thought. See how it's built.

PaperLab runs on your own machine, and every step you just followed is in its code on GitHub.

■ github.com/khannoussi-malek/PaperLab

React · FastAPI · Postgres + pgvector · Redis
Ollama, Anthropic or any OpenAI-compatible model